# Why is programming so difficult to learn? Patterns of Difficulties Related to Programming Learning

**Mid-Stage**

Yorah Bosse
University of São Paulo - USP
Rua do Matão, 1010
CEP 05508-090 – São Paulo – SP - Brazil
+55 11 3722 2998
yorah@ime.usp.br

Marco Aurélio Gerosa
University of São Paulo - USP
Rua do Matão, 1010
CEP 05508-090 – São Paulo – SP - Brazil
+55 11 3091 0753
gerosa@ime.usp.br

DOI: 10.1145/3011286.3011301
http://doi.acm.org/10.1145/3011286.3011301

## ABSTRACT
New software engineers and casual developers are needed in many different areas. However, students face many difficulties while learning the logic of computer programming, frequently failing in university courses. This Ph.D. research aims to identify difficulty patterns related to learning how to program, a crucial part of software engineers training. The research methodology comprises studies that put together results from a systematic literature review and empirical data collected from qualitative and quantitative studies. The difficulties identified will be compiled into a model, which may assist students in sharpening their focus, and teachers in preparing their lessons and teaching material, as well as researchers in employing methods and tools to support learning.

## Categories and Subject Descriptors
[**Theory of Computation**]: Semantics and reasoning – Program Constructs – *Control primitives.*

## General Terms
Human Factors.

## Keywords
Patterns of problems, novice, casual developer, programming, software engineering education.

## 1. INTRODUCTION
Many businesses fail before they are able to fulfil their potential in the market and one of the causes is a lack of software developers [11]. This means it is a serious challenge for modern society to prepare new generations of software developers, since it requires people who are skilled in algorithms and computer programming. Some governments, such as Australia[1], the USA[2], Brazil[3] and the United Kingdom[4], and some organizations, Code.Org[5] for example, are undertaking initiatives in this area with the support of several companies. In addition, some researchers believe that the use of specific software development methods can bring advantages in learn to program, bringing a different, more productive and fun way of teaching. As an example, the research, conducted by Missiroli et al., shows that the precocious exposure of novices programmers to Agile brings advantages not only as a development project, but also as a teaching tool [17].

Programmers are needed to develop and adapt modern systems. However, programming trainees have encountered a number of difficulties. This can be evidenced by the high dropout rates and failures in programming courses [3, 6].

To support researchers in the creation of new methods and development of programming learning systems and the training of new software engineers and developers, this research seeks to identify patterns of difficulties related to learning how to program. The patterns will be independent of programming language. Difficulties, for this research, are all factors that disturb the learning of programming, such as syntax and semantic errors. We focus on the difficulties faced by learners while they are developing the computational thinking for the procedural paradigm. The research questions are as follows:

RQ1 – What is the unsuccessful rate in introduction programming courses?

RQ2 – What difficulties have been reported in the literature with regard to learning how to program?

RQ3 – What are the difficulties of learning how to program from the students' perspective?

RQ4 – What are the difficulties of learning how to program from the instructors' perspective?

RQ5 – What errors in syntax and semantics are recurrently found in the code developed by the students?

RQ6 – How to apply the identified patterns to improve teaching-learning of programming?

## 2. RELATED WORK
"Programming is a complicated business" [15]. This can be seen when evaluating the high percentage of fail presented in Introduction Programming courses [3, 6]. Beaubourg and Mason studied the reasons for high rates, checking, among other factors, the limited problem resolution skills, use of laboratories given for homework, and also the fact that the students go direct to programming, not going through the analysis and design steps [2]. Initiatives to bring programming to schools help to develop skills needed for better performance.

[1] http://mashable.com/2015/09/21/coding-schools-australia/?id=mash-com-fb-aus-link#Yv6gpyKnmGqh
[2] http://www.npr.org/sections/ed/2016/01/12/462698966/the-president-wants-every-student-to-learn-computer-science-how-would-that-work
[3] http://idgnow.com.br/ti-pessoal/2015/04/07/projeto-do-parana-quer-levar-ensino-de-programacao-a-escolas-do-brasil/
[4] http://www.telegraph.co.uk/technology/news/10410036/Teaching-our-children-to-code-a-quiet-revolution.html
[5] Site: https://www.youtube.com/watch?v=nKIu9yen5nc. What Most Schools Don't Teach.

There are several skills needed to learn how to program, being more obvious the ability to solve problems and fundamental knowledge of math. Besides these, Jenkins [15] states that it is necessary to know how to use a computer, to create the program, compile, test, and correct bugs, and learning style and motivation are factors that influence the process of learning to how to program.

Understanding the process of learning a first programming language can help in the task of creating more effective learning environments [13], thereby reducing the difficulties encountered by beginners. Several researchers aimed to find information about these difficulties. Denny et al. [12] show that syntax error is one of the barriers for programming novices, delaying the feedback provided to students about the logic of the code developed. Cechinel et al. reported that the most common problems are the lack of ability to find errors, develop a program to solve a task, and modularization of code using functions and procedures. The topics considered the most difficult were functions and procedures, error handling, and arrays (vectors) [7].

Ribeiro et al. investigated the differences between the use of textual and visual programming environments in the introduction of computer programming [20]. After analyzing the data collected from NASA TLX, activity log, and survey, they concluded that visual programming is a good model for teaching algorithms and programming. Many others researches are conducted to determine if specific methodology, as Agile [17], or code smells by novice programming [14] help to learn how to program.

Lahtinen et al. conducted a survey at six universities in five countries and obtained responses from 559 students and 34 instructors. The answers were given on a scale of 1, easy to learn, to 5, very difficult. As for the educational content covered in the course, the average student perception about how difficulty is the course (mean 2.8) is smaller than instructors (mean 3.5). Students and instructors have the same perceptions of the three content considered more difficult. They are, in this order: pointers, error handling, and recursion. Other contents also considered difficult were: using language library and abstract data types. Both in the view of students and instructors, the three content deemed easier were: selection, repetition, and variables. However, learning the concepts is not considered by students and instructors the biggest problem for programming apprentices. The biggest problem is to apply them in practice [16].

This work contributes to the state of the art identifying patterns of difficulties related to programming learning. As opposed to the traditional focus on syntax problems, our study focuses mainly on the semantic level in the procedural programming paradigm. Other studies cite difficulties, problems, and common errors, however do not provide an in-depth understanding of the difficulties, their relations, and their relevance in multiple scenarios. Thus, knowledge about learning difficulties is spread thin across the literature, and there is little exploration of the problems faced by learners that are not from the computer science area. Additionally, we observed that the majority of the related research predominantly relied on quantitative questionnaire-based methodology. Those that uncovered difficulties missed research questions or objectives related to the in-depth understanding of the phenomena from points of view of students and instructors. In this research, we systematically review the literature and collect data from students and instructors. Our study aims to provide this neglected in-depth understanding of the difficulties and to add to the dominant quantitative survey-based research on learning how to program.

# 3. GOALS AND METHODOLOGY

The main goal of this research is to identify patterns of difficulties related to learning how to program. To achieve this, we will conduct a mixed-method research.

## 3.1 Research Question 1

RQ1 – What is the unsuccessful rate in introduction programming courses?

First of all, it is important to explain the meaning of unsuccessful. For our research, unsuccessful is the result showing that the student has not completed or did not receive a grade necessary to conclude the course. In order to gather evidence about the problem we are dealing with, we will conduct a quantitative study about approvals and failures in introduction programming courses. Much of our data collection will be conducted at the University of São Paulo - USP, which offers annual Introduction Programming courses for thousands of students from several different subject-areas. Thus, our goal is to discover the following: what courses are being offered, what is the profile of the students, what is the failure and drop-out rate, what is the profile of each instructor, and how this compares to the data obtained from the literature. We are also analyzing the possibility to create and submit a survey to several universities from different countries to seek information regarding the unsuccessful rate in introductory programming courses.

Methodology: The first step was to query Introduction to Programming (IP) courses in the academic system using three keywords: "programming," "algorithms," and "computing." Our search returned a total of 207 courses. After analyzing the content of these programs, we selected a group of 31 courses for our research. Only 29 of these courses were considered because two were new, and their classes had not been completed. We obtained an anonymous database which provided the individual results of the 29 courses in the previous five years. We also obtained the school records of each student who attended one of the 29 courses. The preliminary results of the analysis of this database have been shown in two papers [5, 6]. We are currently analyzing the results over a longer period of time and cross referencing additional data such as the results of the students in the university entrance exam and in other subjects. Our aim is to compare the results in the IP course with other courses and specific knowledge areas, such as Languages, Math, Physics, etc.

Validation test plans and publications: Some information has already been obtained , such as the percentage of failures, which corroborated the results obtained by Bennedsen and Caspersen [3]. We will also conduct a quantitative examination of the performance of students at the University of São Paulo, and these will be compared with the results from the literature, and an article will be submitted to a reputable journal.

Threats to validity and other challenges: Some factors may lead to errors in the data disclosed, such as the possibility of errors in the extraction and compilation of the system data. To avoid this, we selected a sample of data for manual checking, and compared this with data from other sources.

Timeline with Milestones from RQ1: Figure 1 below shows the timeline of RQ1.

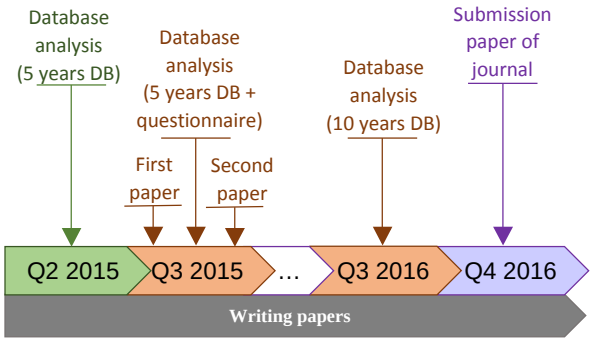


**Figure 1. Timeline with milestones from RQ1.**

## 3.2 Research Question 2

RQ2 – What difficulties have been reported in the literature with regard to learning how to program?

Methodology: A systematic literature review will be carried out to identify difficulties in learning how to program that have been reported and/or empirically investigated so far. We are currently refining the protocol, which includes a search string, databases, and criteria for exclusion/inclusion. On the basis of these factors, the search will be carried out, and this will involve identifying the primary studies, determining what difficulties have been reported and evaluated, and collating the results. A "model of difficulties" will be proposed, in a similar way to a previous study conducted by our research group [21].

Validation test plan and publication: the design of the model will be grounded on the data obtained from the primary studies. We will also compare our results with those of other literature reviews or catalogues, if available. The results will be formatted in an article that will be submitted to a Software Engineering journal.

Threats to validity and other challenges: a threat to validity that we have in mind is the improper definition of the search string. To avoid this threat, we will select articles that are known in the area and the string must return these items in the search results.

Timeline with Milestones from RQ2: Figure 2 below shows the timeline of RQ2.

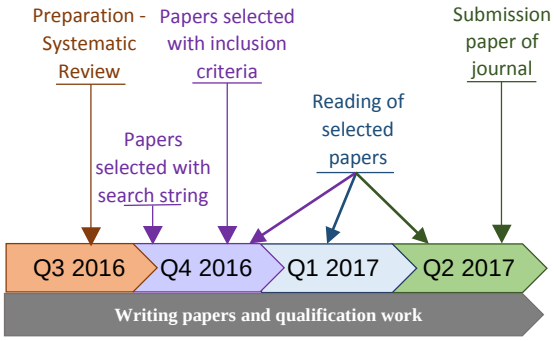


**Figure 2. Timeline with milestones from RQ2.**

## 3.3 Research Question 3

RQ3 – What are the difficulties of learning how to program from the students' perspective?

Methodology: We have been collection information from students by means of three different methods. The first involves individual interviews based on the Think Aloud technique [19]. This technique consists of observing the way users perform specific tasks in controlled environments. The task assigned to the students was made up of 4 exercises with different levels of difficulty. They had to solve a problem using the C programming language, in the Virtual Programming Lab - VPL[6]. VPL is a plugin for Moodle developed by the University of Las Palmas, Canary Islands - ULPGC that offers information about the compilation of the code. In addition, through test cases set by the instructor, it gives feedback to the students about their code. During the interviews, the computer screen and audio was recorded for subsequent analysis. In the pilot study, six students who had failed in the introductory programming course took part in the interviews at the end of 2015.

The second method is based on Diaries [18]. This method was chosen because it enables information about events and experiences to be obtained from the perspective of the subject in a spontaneous way, reducing the time between the occurrence of the event and the time when it is reported to the researchers [4]. In the second half of 2015, students from six courses were invited to participate in our research project by filling out diaries during their studies. They were encouraged to report their experiences, their feelings, the difficulties encountered during their studies, and how they were resolved. 34 students took part in the activity. Google Docs was used for the data collection, by means of individual documents for each student. Open coding and axial coding [8] were used for the data analysis. Our group has already used diaries and this kind of analysis in another situation [22]. This method will be applied again in the second half of 2016 with students from other courses.

The third method will include a survey with specific questions about possible difficulties encountered in the introductory programming course. This survey aims to quantitatively confirm observed patterns and expand the scope of analysis, collect more qualitative and quantitative data. This survey will be applied to students from several universities on two occasions, with classes in the first and second half of 2016.

Validation test plan and publication: data were collected in three different ways and a joint analysis will be carried out to identify the patterns. The data collected with interviews regarding the Think Aloud method were described in a submitted paper and the data from the diaries (part of RQ3) and interviews (part of RQ4) were compiled and a paper is being prepared.

Threats to validity and other challenges: The greatest challenge is to persuade the students to participate by filling in the diaries and answering the questions in the survey. We will go to some classrooms and collect the responses in person.

## 3.4 Research Question 4

RQ4 – What are the difficulties of learning how to program from the instructors' perspective?

Methodology: Interviews were conducted in late 2015 with 16 instructors involved in the Introduction to Programming course. Ten instructors were randomly selected and the other 6 were those that were teaching Introduction Programming courses that semester. The purpose of the interviews was to find out what are the difficulties of the students in the view of the instructor. Inquiries were made about the syllabus of the subject to determine the difficulties observed by the instructors. The interviews were conducted individually, the content was recorded on audio and transcribed. Currently, we are at the stage of analyzing and formatting data employing the methodology of Grounded Theory.

With also aim to conduct surveys to collect additional data and confirm some hypothesis raised during the analysis.

Validation test plan and publication: data from the diaries (part of RQ3) and interviews (part of RQ4) were formatted and will be presented in a paper. Data collected from students (RQ3), together with the data from the instructors (RQ4) will be analyzed and formatted in a paper that will be submitted to an international journal.

Timeline with Milestones from RQ3 and RQ4: Figure 3 below shows the timeline of RQ3 and RQ4.

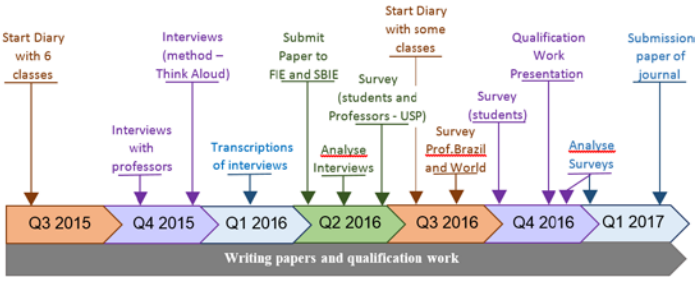


**Figure 3. Timeline with milestones from RQ3 and RQ4.**

[6] Virtual Programming Lab – VPL. URL: http://vpl.dis.ulpgc.es/index.php/about

## 3.5 Research Question 5

RQ5 – What errors in syntax and semantics are recurrently found in the code developed by the students?

Methodology: Code made by students during the semesters has been collected for analysis of error patterns. We will connect these patterns to those identified from the previous RQs. We will use mining software repositories techniques in order to collect, clean, and analyze the data, searching for the patterns.

Validation test plan and publication: A problem must be detected in at least three different situations in order to be considered a pattern. We plan to gather evidence of the reported difficulties and find new patterns analyzing the source code produced by the learners.

Threats to validity and other challenges: The analysis of syntax errors can be done by a system that analyzes the code submitted by the students. The analysis of the logic errors is more complicated to be performed by the system. We are still testing different way of doing this activity.

Timeline with Milestones from RQ5: Figure 4 below shows the timeline of RQ5.

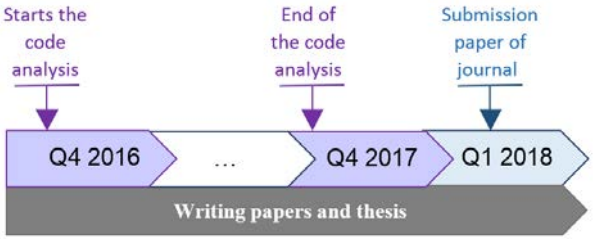


**Figure 4. Timeline with milestones from RQ5.**

## 3.6 Patterns Definition

Based on the results of the RQ2 to RQ5, we will compile the difficulties observed into patterns. Each pattern will comprise a name, situation in which it occurs, how to solve it, and examples. We will also categorize the patterns according to the Bloom's taxonomy. Bloom created categories for educational goals[1] (Figure 5). Each category has a set of action words that could be used help identify the kind of knowledge related to the difficulties.

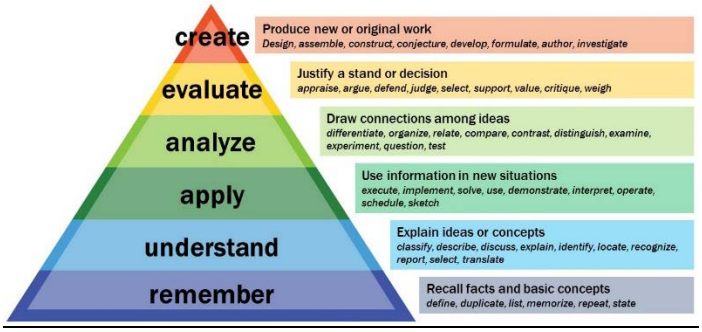


**Figure 5. Bloom's taxonomy[7].**

## 3.7 Research Question 6

RQ6 – How to apply the identified patterns to improve teaching-learning of programming?

Methodology: Guided by the previously modeled patterns, we will follow an action research approach. We will specify the teaching strategy for each pattern of difficulty identified, according to what the instructors reported. After this, we will define two groups learning the same contents (two different courses or two group inside the same course). In one of them, we will apply the strategy. In the other one, we will analyze the students manifest the difficulty related to the pattern (Figure 6). At each stage of action research, different elements of the model will be evaluated. We will perform the triangulation of data to validate the results and improve accuracy [9, 10].


**Figure 6. Pattern and strategy validation process.**

Validation test plan and publication: An article describing the research and its results will be submitted for publication in an international journal.

Threats to validity and other challenges: One challenge will be to have classes and instructors enough to work in this action research. Another challenge will be to have time enough to make all validation.

Timeline with Milestones from RQ6: Figure 7 below shows the timeline of RQ6.

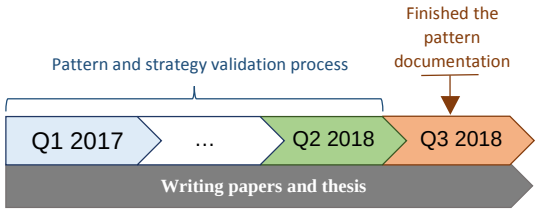


**Figure 7. Timeline with milestones from RQ6.**

## 4. PROGRESS AND NEXT STAGES

In the following, we present some results achieved.

RQ1 – What is the unsuccessful rate in introduction programming courses?

We have analyzed a database comprising results from introductory programming courses at the University of São Paulo for the years 2010-2014. Our results corroborate those of other studies [3]. Out of the 18,784 registrations made in the analyzed period, 30% resulted in failures or dropouts, what was fairly constant over the years. We evidenced a higher failure rate for students who were not from the computer science area, reaching 30.3% compared to 25.1% of students who are from the area. We also found that more than 25% of who were approved attended two

[7] https://cft.vanderbilt.edu/guides-sub-pages/blooms-taxonomy/

or more times the course. This course is among the ones with the highest failure rates.

RQ3 – What are the difficulties of learning how to program from the students' perspective?

In the second half of 2015, 34 students from six courses filled diaries about their studies. They reported difficulties and some strategies found to solve them. In the following, we present some students' comments identifying who wrote them by means of a subscript "a" followed by a numbering. The data found in these diaries were analyzed using Grounded Theory procedures and they were grouped by concepts, forming four categories: Difficulties, Study Strategies, Preferences, and Self-assessments. In the following, we present some results from the first category. We detected that 'syntax error', with 13 occurrences, was the problem most frequently reported by students, with comments like: "*I still have a lot of errors in basic things like braces, parentheses, and semicolons*"$_{a1}$, "*the program still didn't execute due to some syntax errors that I don't know how to solve*"$_{a20}$ and, "*It is returning syntax error all the time*"$_{a22}$. This type of error makes students to return often to the code before being able to check if their logic was correct.

Problems with 'variables' was the second most cited, as noted in the following comment "*I had difficulty to understand what should be float and what should be int type, so I had to go testing to find*"$_{a1}$. The concept 'Language + IDE + Error Message' was also widely cited, having complaints as: "*initially, I had difficulty with the language, even with the complementary material, I had difficulty putting into practice*"$_{a5}$, "*because the program's messages did not help at all*"$_{a20}$ and, "*I could not interpret the messages that the program showed, so I had to execute parts of the program separately in another window until I could identify the error*"$_{a20}$. In addition to these complaints about the language and the error messages, we received comments related to the IDE, as "*the instructor's site doesn't have the link to download the updated version of the IDE, and the available version doesn't work on Windows 8*"$_{a17}$.

In an another study, aimed at getting more information about the students and their behavior during the studies, using the Think Aloud method, we conducted interviews with six students, lasting about one hour each. These students did not succeed during the semester and needed to make the final test if they wanted to be approved. During the interviews, they were challenged to solve four exercises with increasing degree of difficulty. Their interview session was registered, including the computer screen and audio recordings, for analysis.

One of the observed attitudes, adopted by 2 of the students, was to take notes while they read the statements (student 1 and 3). These 2 had no better results than the others, but one of them, when asked by the interview moderator, informed that "*annotating helps to remember what needs to be done because otherwise I cannot remember*". Analyzing the behavior of the respondents while running the session, we observed that this annotation process helped, for example, in the definition of which and how many variables were required to solve the task. One difference between these students and the others is that they had less mistakes in declaring the variables and setting their types, practically they did not need to go back to the code to change what they had written.

The interview moderator observed in two students a reaction while reading the statement. Student 6 had not read the entire statement when he stopped reading to make the comment "*I get nervous when I see the word matrix*". The student 1, when started to read the second question spoke instantly "*I do not like function*" and "*I have difficulties with function parameters*". Student 1 said "*At a first glance I dislike this exercise, I like exercises that have numbers*". In these three situations the students did not succeed on solving the exercise. This may be a sign that the students create a barrier to the content that they face more difficulty.

We also noticed uncertainty in students and some degree of absence of analytical thinking. They are used to copy and paste the code to read matrix elements, but when faced by compilation errors, there stated comments like "*We will see now. Must be something wrong. There is always something wrong.*" The moderator noted that the commands to which they referred to were correctly written, but with undeclared name. Moreover, in some moments, the student faced problems with intention and practice. They verbalized something, but wrote something different. This situation was detected during interviews and can be observed in the comments "*I do not know if it's like this to read an array, but okay*"$_{a1}$ and "*I think something is missing in this print*"$_{a6}$.

Syntax errors were common in all the interviews and exercises, e.g. opening and closing structures with brackets, colons, correct spelling of the commands, among others. Some errors are noteworthy, such as: (A) attempt to read the data in the matrix; (B) create an unnamed function, besides the incorrect declaration of the variables to receive the parameters, and (C) semi-colon ending a structure of repetition and selection that has not even started.

When semantic errors occurred, students usually became more disappointed than with syntax errors. With syntax errors, they seemed to be more accustomed. The semantic errors made students to drop out the exercise faster, because they already realized that they need more time to fix semantic errors.

RQ4 – What are the difficulties of learning how to program from the instructors' perspective?

We randomly selected 14 instructors of the Computer Science Department of the University of São Paulo, that taught introductory programming. Individual interviews were conducted with each of them. The interviews were recorded and are being analyzed using Grounded Theory procedures. The objective of this study is to seek the difficulties encountered by students in the instructors' view.

The main difficulty, cited by instructors, is the 'logical reasoning'. They have tried pseudocode, but most have given up. Some instructors use the pseudocode only to quickly explain the concept, then they go straight to the programming language. Others use pseudocode in parallel, i.e., they develop in pseudocode and then translate into the programming language: "*...if you don't know where to start, writes in natural language a draft. After this, you go to the pseudocode and only at the end you go to Python*"$_{p3}$. They also reported that the experience sometimes makes it difficult to teach: "*I see a problem, it already is structured in my mind and I don't know how it happens*"$_{p1}$.

About the 'syntactical issues of language', they all agreed that C syntax has much more details to be observed during programming. It was also commented that the 'choice of language' influences on the development of the student.

They cited operators - arithmetic, logical and relational – as sources of difficulties. Students get confused with precedence. There is also difficulty in differentiating the logical operators 'and' and 'or' and do arithmetic with variables from the same type, but resulting in a different type. An example is when the division of two integers results zero, as the division of 1 by 2. To display the correct result, the type of the resulting value must be float, "*it is hard to them realize the error*"$_{p4}$.

Among the structures of selection/decision and repeat/loop, most instructors start teaching the loop structure, more specifically by 'while'. It was often cited that 'while' gives the impression of having more control about the structure, that the student prefers 'while' rather than 'for', information that corroborates those of diaries written by students. Students also have difficulty in embedded loops and how to set the break condition. In the selection structure, the difficulty is 'see the if..else pairs'. In addition, the students mix concepts between decision and loop structures.

For arrays, there is the 'forgotten to put the index' regarding the position, which is solved with the strategy of 'intensive practice': "*You have to do by repeating, which is a tiring business at the beginning*"$_{p1}$. Instructors also commented that students understand the concept, but fail to apply in practice, information that reinforces what has already been published [23].

About function, the difficulty lies in understanding the scope of the variables and the importance of the return value. Teachers believe that there are not major problems with parameter passing, however, when it is by reference and the language used is C, the difficulty increases.

Instructors comment that there are some factors that help to make difficult to teach how to program, as: heterogeneity of the groups and between groups, low participation in class, low frequency, very large classes, disinterest in learning, the programming language adopted, trauma of students that repeat the course, among others. They also expressed concern about trying to motivate the student. They use strategies like working with games, challenge, and competition. Instructors complained about trying to know by heart instead of learning. This was a strategy also quoted by the students in the diaries.

## 5. NEXT STEPS

The next steps of the research are:

1. Complete the database analysis about the last 10 years of the Introduction to Programming course at USP (RQ1).
2. Perform the systematic literature review to find difficulties reported (RQ2).
3. Apply the technique of diaries in more courses and run a confirmatory questionnaire with students (RQ3).
4. Complete interviews with instructors from USP and apply an extended survey for instructors from outside. Analyze and tabulate the data that will give us information about the students' difficulties perceived by teachers (RQ4).
5. Consolidate the results from the systematic literature review and the data collection in a single model
6. Analyze the source code produced by students (RQ5).
7. Validate the patterns (RQ6).

The specific timelines were presented in the method section.

## 6. CONCLUSION

Until now, we are not a lot of results, but some patterns are already defined. One of these is the difficulty that students have to work with functions. Understanding the scope of variables and why it is necessary to pass and return parameters is not easy for them. Some strategies used by instructors to mitigate this barrier were explained in the interviews with instructors.

We expected that the patterns of difficulty related to programming learning help students in their studies, teachers in preparing their lessons, and researchers in developing new tools to support teaching and learning programming. This will help to train the next generation of software engineers.

## 7. ACKNOWLEDGMENTS

We would like to express our thanks to the instructors and students from the University of São Paulo for their valuable assistance with our research project.

## 8. REFERENCES

[1] Anderson, L.W.. et al. 2001. *A taxonomy for learning, teaching, and assessing : a revision of Bloom's taxonomy of educational objectives*. Longman.

[2] Beaubouef, T. and Mason, J. 2005. Why the high attrition rate for computer science students. *ACM SIGCSE Bulletin*. 37, 2 (2005), 103.

[3] Bennedsen, J., & Caspersen, M.E. 2007. Failure rates in Introductory Programming. *ACM SIGCSE Bulletin*. 39, 2 (2007), 32–36.

[4] Bolger, N. et al. 2003. Diary methods: Capturing life as it is lived. *Annual Review of Psychology*. 54, (2003), 579–616.

[5] Bosse, Y. and Gerosa, M.A. 2015. As Disciplinas de Introdução à Programação na USP: um Estudo Preliminar. *WAlgProg - I Workshop de Ensino em Pensamento Computacional, Algoritmos e Programação*. Cbie (2015), 1389.

[6] Bosse, Y. and Gerosa, M.A. 2015. Reprovações e Trancamentos nas Disciplinas de Introdução à Programação da Universidade de São Paulo : Um Estudo Preliminar. *WEI - Workshop sobre Educação em Computação*. (2015), 1–10.

[7] Cechinel, C. et al. 2008. Desenvolvimento de Objetos de Aprendizagem para o Apoio à Disciplina de Algoritmos e Programação. *Simpósio Brasileiro de …*. (2008).

[8] Corbin, J. and Strauss, A. 1990. Grounded theory research: Procedures, canons, and evaluative criteria. *Qualitative Sociology*. 13, (1990), 3–21.

[9] Creswell, J.W. 2013. *Research design: Qualitative, quantitative, and mixed methods approaches*.

[10] Creswell, J.W. and Clark, V.L.P. 2007. *Designing and conducting mixed methods research*.

[11] Crowne, M. 2002. Why software product startups fail and what to do about it. Evolution of software product development in startup companies. *IEEE International Engineering Management Conference*. 1, (2002), 338–343.

[12] Denny, P. et al. 2011. Understanding the syntax barrier for novices. *Proceedings of the 16th ACM conference on Innovation and technology in computer science education - ITiCSE '11*. (2011), 208.

[13] Garner, S. et al. 2005. My program is correct but it doesn't run: A preliminary investigation of novice programmers' problems. *Conferences in Research and Practice in Information Technology Series*. 42, (2005), 173–180.

[14] Hermans, F. and Aivaloglou, E. 2016. Do Code Smells Hamper Novice Programming ? (2016).

[15] Jenkins, T. 2002. On the Difficulty of Learning to Program. *ICS - International Conference on Supercomputing*. (2002).

[16] Lahtinen, E. et al. 2005. A study of the difficulties of novice programmers. *ACM SIGCSE Bulletin*. 37, 3 (2005), 14–18.

[17] Missiroli, M. et al. 2016. Learning Agile Software Development in High School : an Investigation. *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. (2016), 293–302.

[18] Reis, H.T. 1994. Domains of experience: investigating relationship processes from three perspectives. 87–110.

[19] Renzi, A.B. et al. 2012. Use of Think-Aloud Protocol to Verify Usability Problems and Flow During Use of Entertainment and Personal Journal. *12o Congresso Internacional de Ergonomia e Usabilidade de Interfaces Humano-Computador* (Natal - Brasil, 2012), 7.

[20] Ribeiro, R. da S. et al. 2014. Programming web-course analysis: How to introduce computer programming? *2014 IEEE Frontiers in Education Conference (FIE) Proceedings*. 2015–Febru, February (2014), 1–8.

[21] Steinmacher, I. et al. 2015. A systematic literature review on the barriers faced by newcomers to open source software projects. *Information and Software Technology*. 59, (2015), 67–85.

[22] Steinmacher, I. et al. 2016. Overcoming open source project entry barriers with a portal for newcomers. *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. (2016), 273–284.

[23] Winslow, L.E. 1996. Programming Pedagogy - A Psychological Overview. *ACM SIGCSE Bulletin*. 28, 3 (1996), 17–22.